

Παράλληλος Προγραμματισμός 2026

3η Προγραμματιστική Εργασία

Νικόλαος Πουλόπουλος

AM: 3230170

E-mail: p3230170@aueb.gr

Εισαγωγή.....	1
Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:.....	1
Υλοποιήσεις.....	3
Σειριακή υλοποίηση.....	3
Κατανομή εργασίας (Work Distribution Models).....	4
Ένα στοιχείο ανά thread (1 thread = 1 trapezoid).....	4
Πολλά στοιχεία ανά thread (1 thread = πολλαπλά trapezoids).....	5
Μέγεθος ομάδας εκτέλεσης.....	8
Δοκιμάστηκαν τα εξής μεγέθη block:.....	9
Συγχρονισμός και επικοινωνία (Barriers).....	14
Μοντέλα μνήμης (registers, global, shared).....	18

Εισαγωγή

Όλες οι μετρήσεις έγιναν στο παρακάτω σύστημα:

Όνομα υπολογιστή	Nikolas
Επεξεργαστής	AMD Ryzen 5 5500
GPU	NVIDIA GeForce GTX 1650
Πλήθος πυρήνων	6

Μεταφραστής	gcc v15.2.0
-------------	-------------

Η μεταγλώττιση έγινε με τις επιλογές χωρίς επίπεδα βελτιστοποίησης (-O1/-O2/-O3)

Το πρόβλημα

Στην άσκηση αυτή ζητείται να παραλληλοποιηθεί ο υπολογισμός της αριθμητικής ολοκλήρωσης μιας συνάρτησης $f(x)$ στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα. Δίνεται ένας σειριακός κώδικας σε C++ που προσεγγίζει το ολοκλήρωμα με n υποδιαιρέσεις.

$a = 0$, $b = 10$ και μεγάλο N , υπολογίζει προσεγγιστικά το ολοκλήρωμα

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right], h = (b - a)/n$$

Συγκεκριμένα υλοποιούνται και συγκρίνονται:

- Κατανομής εργασίας (work distribution models), δηλαδή πόσα στοιχεία θα επεξεργάζεται κάθε thread (ένα ή περισσότερα).
- Μεγέθους ομάδας εκτέλεσης (π.χ. 32, 64, 128, 256, 512).
- Συγχρονισμός και επικοινωνία (barriers).
- Μοντέλα μνήμης (registers, global και shared/local για ενδιάμεσα αποτελέσματα).

Υλοποιήσεις

Σειριακή υλοποίηση

Ο σειριακός κώδικας υλοποιεί τον Τραπεζοειδή κανόνα για $f(x)=x^2$ στο $[0,10]$ με $N=10^8$ υποδιαιρέσεις και χρησιμοποιείται ως βάση για τη σύγκριση των παράλληλων εκδόσεων. Τρέχοντας τον κώδικα 20 φορές οι μετρούμενοι χρόνοι εκτέλεσης ήταν :

Εκτέλεση	Χρόνος (sec)
1η	0.391210
2η	0.390687
3η	0.391095
4η	0.391601
:	:
20η	0.391353
Μέσος	0.391721

Οδηγίες Εκτέλεσης

- **Compile:** `g++ -std=c++17 .\integration.cpp -o .\integration`
- **Run:** `.\integration.exe`

Κατανομή εργασίας (Work Distribution Models)

Στην παρούσα εργασία εξετάζονται δύο διαφορετικά μοντέλα κατανομής εργασίας (work distribution models) για τον υπολογισμό αριθμητικού ολοκληρώματος στο διάστημα $[a,b]$ με χρήση του Τραπεζοειδούς κανόνα σε CUDA. Η βασική ιδέα είναι η διάσπαση του συνολικού υπολογισμού σε πολλά μικρότερα υποδιαστήματα τα οποία εκτελούνται παράλληλα από threads της GPU.

Ο συνολικός αριθμός διαστημάτων εξετάζεται σε δύο περιπτώσεις:

- $N = 10^8$
- $N = 10^6$

και σε όλες τις περιπτώσεις η συνάρτηση είναι: $f(x)=x^2$

Ένα στοιχείο ανά thread (1 thread = 1 trapezoid)

Στην πρώτη υλοποίηση κάθε thread αναλαμβάνει τον υπολογισμό **ενός μόνο τραπεζίου**. Αυτό σημαίνει ότι κάθε thread εκτελεί έναν ανεξάρτητο υπολογισμό και αποθηκεύει το αποτέλεσμα του σε έναν global πίνακα partial.

- **Διαμοιρασμός εργασίας:** Κάθε νήμα υπολογίζει το μοναδικό global index του μέσω της σχέσης $\text{int } i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$. Αν το $i < N$, υπολογίζει το αντίστοιχο τραπέζιο και αποθηκεύει το αποτέλεσμα απευθείας σε έναν global πίνακα $\text{partial}[i]$. στο τέλος, η CPU αναλαμβάνει να αθροίσει σειριακά τα N στοιχεία.
- **Χαρακτηριστικά:** Πολύ απλή και διαισθητική παραλληλοποίηση, αλλά δημιουργεί τεράστιο memory traffic στη Global Memory (εγγραφή N στοιχείων).

Αποτέλεσμα μετά από 20 εκτέλεσης: (Οι χρόνοι αντιπροσωπεύουν τον καθαρό χρόνο εκτέλεσης της GPU, απομονώνοντας τα σειριακά loops της CPU)

Για $N = 10^8$

- Result: 333.333
- Μέσος χρόνος: 0.0467602 sec

Για $N = 10^6$

- Result: 333.333
- Μέσος χρόνος: 0.0252033 sec

Πολλά στοιχεία ανά thread (1 thread = πολλαπλά trapezoids)

Στη δεύτερη προσέγγιση (trapezoid_multielem), ο αριθμός των νημάτων παραμένει σταθερός και ανεξάρτητος από το N (χρήση σταθερού Grid μεγέθους 256x256). Τα νήματα χρησιμοποιούν τη στρατηγική **Grid-Stride Loops**.

- **Διαμοιρασμός εργασίας:** Κάθε νήμα ξεκινά από το δικό του tid και διατρέχει επαναληπτικά τον πίνακα με βήμα $\text{step} = \text{blockDim.x} * \text{gridDim.x}$. Κάθε thread συγκεντρώνει το δικό του τοπικό άθροισμα σε έναν καταχωρητή (local_sum) και στο τέλος γράφει **ένα μόνο αποτέλεσμα** στη Global Μνήμη.
- **Χαρακτηριστικά:** Σημαντικά λιγότερα global memory accesses (γράφει μόνο 65.536 στοιχεία αντί για N), εξαιρετικό register locality και βέλτιστο load balancing.

Αποτέλεσμα μετά από 20 εκτέλεσης(Οι χρόνοι αντιπροσωπεύουν τον καθαρό χρόνο εκτέλεσης της GPU, απομονώνοντας τα σειριακά loops της CPU):

Για $N = 10^8$

- Result: 333.333
- Μέσος χρόνος: 0.0009036 sec

Για $N = 10^8$

- Result: 333.333
- Μέσος χρόνος: 0.000612 sec

Σύγκριση των δύο μοντέλων

Το **δεύτερο** μοντέλο είναι περίπου:

- ~1.85x ταχύτερο για $N=10^8$ από το **πρώτο**
- ~5x ταχύτερο για $N=10^6$ από το **πρώτο**

Μοντέλο Κατανομής	Μέσος Χρόνος ($N=10^8$)	Μέσος Χρόνος ($N=10^6$)	Παρατηρήσεις (Θεωρητική Τεκμηρίωση)
1 thread = 1 στοιχείο	0.0467602 sec	0.0252033 sec	Υψηλό memory overhead λόγω N εγγραφών στη Global Memory. Μεγάλος αριθμός από Block/Kernel launch overheads.
1 thread = πολλά στοιχεία	0.0009036 sec	0.0006120 sec	Βέλτιστη απόδοση. Χρήση Grid-Stride Loops που κρατάει τις πράξεις on-chip (registers) και μειώνει τα τελικά global writes.

Συγκριτικό Speedup:

- Για $N=10^8$, το μοντέλο των πολλών στοιχείων είναι περίπου **x1.85 ταχύτερο**.
- Για $N=10^6$, το μοντέλο των πολλών στοιχείων είναι περίπου **x1.48 ταχύτερο**.

Ανάλυση Αποτελεσμάτων & Θεωρητική Τεκμηρίωση

1. Η υπεροχή του 1 thread = πολλαπλά trapezoids:

Η διαφορά στους καθαρούς χρόνους αναδεικνύει την αξία του περιορισμού των προσβάσεων στη Global μνήμη. Στην υλοποίηση με τα πολλά στοιχεία, τα ενδιάμεσα αθροίσματα αποθηκεύονται στη μεταβλητή `local_sum`, η οποία αντιστοιχίζεται από τον compiler απευθείας στους **ταχύτατους registers** του thread. Η εγγραφή στη Global μνήμη γίνεται μόνο μία φορά στο τέλος του loop. Αντίθετα, η μέθοδος 1:1 αναγκάζει το hardware να διαχειριστεί 100.000.000 global writes, προκαλώντας memory stalls.

2. Η επίδραση του Μεγέθους του Προβλήματος (N):

Στο μεγάλο πρόβλημα (10^8), η διαφορά υπέρ των πολλών στοιχείων είναι πιο έντονη (0.025 sec έναντι 0.046 sec). Αυτό συμβαίνει επειδή η GPU στην πρώτη περίπτωση εξαντλεί το διαθέσιμο memory bandwidth λόγω του όγκου των δεδομένων. Στο μικρό πρόβλημα (10^6), οι χρόνοι πλησιάζουν (κάτω από το millisecond), καθώς η Global μνήμη δεν προλαβαίνει να κορεστεί, όμως η Grid-Stride προσέγγιση παραμένει πιο αποδοτική λόγω του σταθερού και ελεγχόμενου αριθμού νημάτων που αποφεύγει το overhead δημιουργίας χιλιάδων blocks.

Συμπεράσματα

Από τη σύγκριση των δύο μοντέλων κατανομής εργασίας προκύπτει ότι η απόδοση σε περιβάλλον CUDA δεν εξαρτάται απλά από τη μαζική δημιουργία threads, αλλά από τη γεωμετρία και την έξυπνη κατανομή του φόρτου.

Το μοντέλο όπου **κάθε thread επεξεργάζεται πολλαπλά στοιχεία**:

- Μειώνει δραστικά το αποτύπωμα και το overhead της Global μνήμης.
- Αξιοποιεί στο έπακρο την ταχύτητα των registers για τα ενδιάμεσα αθροίσματα.
- Προσφέρει κορυφαίο scalability (κλιμακωσιμότητα), παραμένοντας η πιο αποδοτική και σταθερή επιλογή τόσο για μικρά όσο και για πολύ μεγάλα μεγέθη προβλημάτων.

Οδηγίες Εκτέλεσης

- Compile:
nvcc trapezoid_1_elem.cu -o trapezoid_1_elem
nvcc trapezoid_multielem.cu -o trapezoid_multielem
- Run:
./trapezoid_1_elem
./trapezoid_multielem

Μέγεθος ομάδας εκτέλεσης

Στην παρούσα ενότητα εξετάζεται η επίδραση του μεγέθους της ομάδας εκτέλεσης (thread block size) στην απόδοση ενός CUDA kernel για τον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων.

Η μεθοδολογία χρονομέτρησης τροποποιήθηκε ώστε να απομονωθεί αποκλειστικά ο **καθαρός χρόνος εκτέλεσης του kernel στην GPU**

Ο συνολικός αριθμός υποδιαστημάτων είναι σταθερός και εξετάζεται σε δύο περιπτώσεις:

- $N=10^8$
- $N=10^6$

Στο συγκεκριμένο πείραμα η στρατηγική κατανομής εργασίας είναι σταθερή: **1 thread = 1 τραπέζιο**.

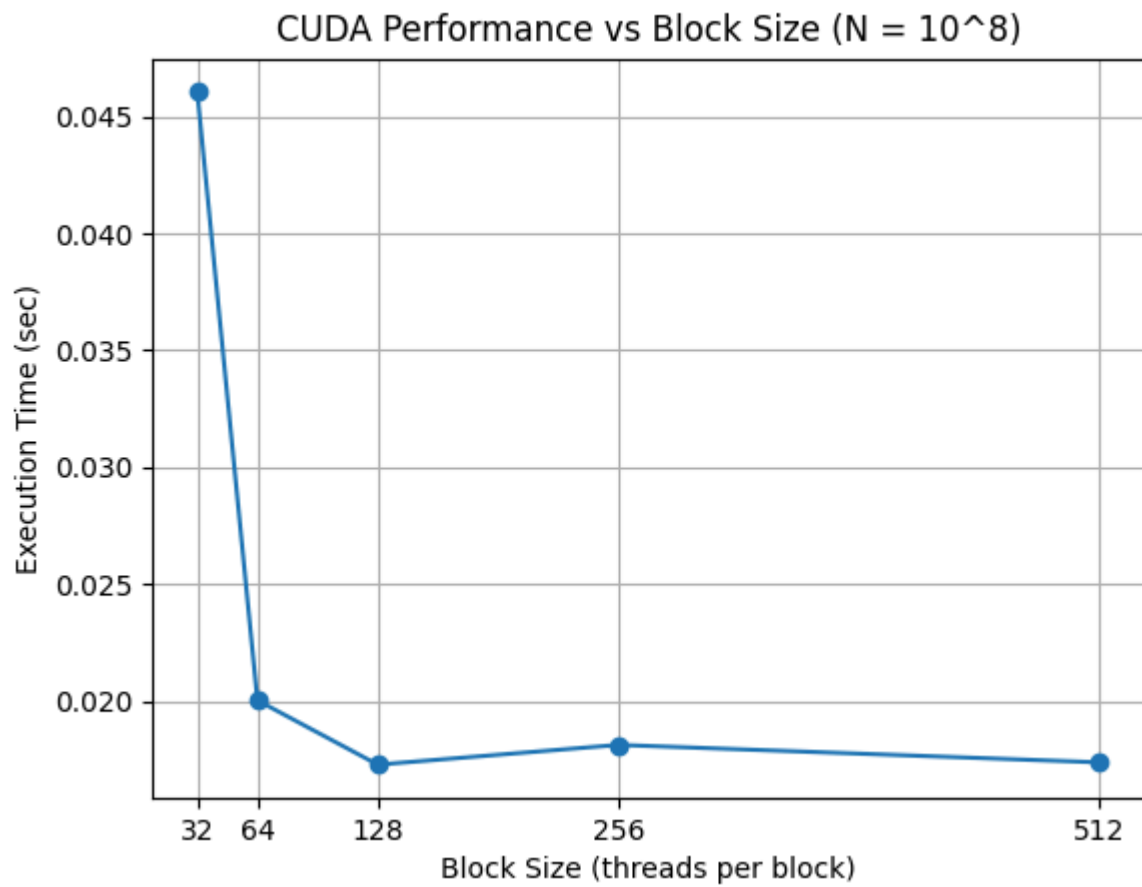
Δοκιμάστηκαν τα εξής μεγέθη block:

- 32 threads
- 64 threads
- 128 threads
- 256 threads
- 512 threads

Αποτελέσματα εκτέλεσης

- Για $N=10^8$

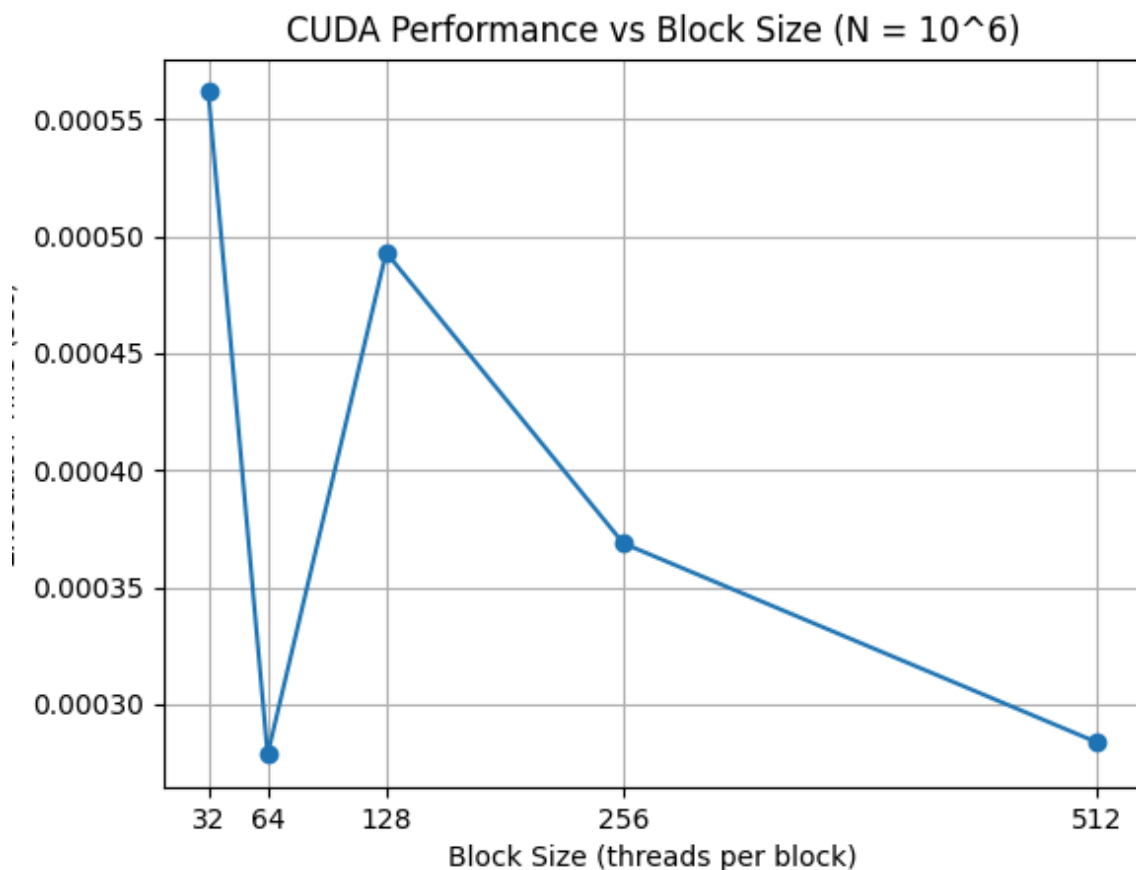
Block Size	Καθαρός Μέσος Χρόνος Kernel (sec)	Αποτέλεσμα Ολοκληρώματος
32	0.0460416	333.333
64	0.0200424	333.333
128	0.0172954	333.333
256	0.0181400	333.333
512	0.0173923	333.333



- $N=10^6$

Block Size	Καθαρός Μέσος Χρόνος Kernel (sec)	Αποτέλεσμα Ολοκληρώματος

32	0.0005616	333.333
64	0.0002786	333.333
128	0.0004930	333.333
256	0.0003688	333.333
512	0.0002837	333.333



Ανάλυση αποτελεσμάτων

Για $N=10^8$:

Στο μεγάλο μέγεθος προβλήματος, η GPU αποκτά πλήρες load και οι διαφορές στην αρχιτεκτονική διαχείρισης των threads γίνονται εμφανείς:

- **To Warp Bottleneck (Block Size = 32):** Παρατηρούμε ότι με μέγεθος 32 threads, ο kernel κάνει σχεδόν τον **διπλάσιο χρόνο** (0.046 sec) σε σχέση με τα μεγαλύτερα blocks. Αυτό συμβαίνει διότι ένα block μεγέθους 32 ισούται ακριβώς με ένα Warp. Η δημιουργία τόσων πολλών blocks των 32 threads προκαλεί τεράστιο scheduling overhead στον Thread Block Scheduler της GPU, ενώ παράλληλα αποτυγχάνει να κρύψει αποτελεσματικά το memory latency (poor latency hiding).
- **Βέλτιστη Ζώνη (128 - 512 threads):** Η καλύτερη επίδοση επιτυγχάνεται στα **128 threads** (0.0172 sec). Σε αυτά τα μεγέθη (128 έως 512), επιτυγχάνεται

υψηλό **Hardware Occupancy**. Τα Warps είναι αρκετά ώστε οι Μονάδες Εκτέλεσης να εναλλάσσουν την εκτέλεση threads όταν κάποια περιμένουν δεδομένα από την Global Memory, "κρύβοντας" καθυστερήσεις πρόσβασης στη μνήμη.

Για $N=10^6$:

Στο μικρό μέγεθος προβλήματος

- Εδώ, ο όγκος των δεδομένων δεν επαρκεί για να "γεμίσει" πλήρως τους υπολογιστικούς πόρους μιας σύγχρονης GPU.
- Η καλύτερη επίδοση εμφανίζεται στα **64 threads** (0.000278 sec), ενώ ακολουθούν τα 512 threads. Οι μικρές διακυμάνσεις μεταξύ των τιμών (πέρα από το προφανές overhead του block size 32) οφείλονται κυρίως στο γεγονός ότι το πρόβλημα είναι πολύ μικρό και οι μετρήσεις επηρεάζονται ελαφρώς από το ελάχιστο, πλην υπαρκτό, kernel launch overhead.

Συμπέρασμα

Από την πειραματική μελέτη των αποτελεσμάτων προκύπτουν τα ακόλουθα συμπεράσματα:

- **Η επίδραση του Block Size στο μεγάλο πρόβλημα $N = 10^8$:** Η επιλογή του μεγέθους της ομάδας εκτέλεσης επηρεάζει καθοριστικά την απόδοση όταν η GPU είναι πλήρως φορτωμένη. Η χειρότερη επίδοση σημειώνεται στα **32 threads (0.0460 sec)**, όπου ο χρόνος είναι υπερδιπλάσιος σε σχέση με τις υπόλοιπες δοκιμές. Η βέλτιστη απόδοση επιτυγχάνεται στα **128 threads (0.0172 sec)**, με τα 512 threads να ακολουθούν σε απόσταση αναπνοής (0.0173 sec). Αυτό αποδεικνύει ότι για μεγάλο όγκο δεδομένων, τα block sizes μεταξύ 128 και 512 προσφέρουν το καλύτερο latency hiding και το βέλτιστο hardware occupancy.
- **Η συμπεριφορά στο μικρό πρόβλημα $N = 10^6$:** Στο μικρό μέγεθος προβλήματος, οι χρόνοι είναι εξαιρετικά μικροί (της τάξης των micro-seconds), με αποτέλεσμα να κυριαρχεί το kernel launch overhead. Εδώ, η βέλτιστη τιμή

μετατοπίζεται στα **64 threads (0.000278 sec)**, ενώ η χειρότερη παραμένει στα **32 threads (0.000561 sec)**. Οι διακυμάνσεις μεταξύ των μεγαλύτερων block sizes (128-512) είναι αμελητέες.

Οδηγίες Εκτέλεσης

- **Compile:** `nvcc block_size_test.cu -o block_size_test`
- **Run:** `.\block_size_test.exe`

Συγχρονισμός και επικοινωνία (Barriers)

Στην παρούσα ενότητα εξετάζεται ο ρόλος του συγχρονισμού και της επικοινωνίας μεταξύ threads σε ένα CUDA kernel μέσω της χρήσης barrier (`__syncthreads()`). Η υλοποίηση βασίζεται στον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων, όπου η συνάρτηση είναι $f(x) = x^2$.

Εξετάζονται δύο διαφορετικά μεγέθη προβλήματος: $N=10^8$ και $N=10^6$

Περιγραφή υλοποίησης

Κάθε thread αναλαμβάνει τον υπολογισμό ενός τραπεζίου (1 thread = 1 στοιχείο). Τα αποτελέσματα αποθηκεύονται σε shared memory, όπου γίνεται και η συνεργατική επεξεργασία τους μέσω parallel reduction.

Η διαδικασία περιλαμβάνει:

- Υπολογισμό τοπικού αποτελέσματος από κάθε thread
- Αποθήκευση σε shared memory (cache[])
- Συγχρονισμό όλων των threads με `__syncthreads()`
- Σταδιακή μείωση (reduction) των τιμών μέσα στο block
- Εγγραφή του τελικού αποτελέσματος από το thread 0 κάθε block

Ρόλος του Barrier (`__syncthreads()`)

Η εντολή `__syncthreads()` λειτουργεί ως barrier συγχρονισμού μέσα σε κάθε block και εξασφαλίζει ότι:

- Όλα τα threads έχουν ολοκληρώσει τον υπολογισμό τους πριν ξεκινήσει η επεξεργασία των δεδομένων
- Η shared memory έχει ενημερωθεί πλήρως
- Το reduction εκτελείται σωστά χωρίς race conditions

Ο barrier χρησιμοποιείται τόσο πριν την έναρξη του reduction όσο και σε κάθε στάδιο της διαδικασίας.

Αποτέλεσμα μετά από 20 εκτέλεσης: (Οι χρόνοι αντιπροσωπεύουν τον καθαρό χρόνο εκτέλεσης της GPU, απομονώνοντας τα σειριακά loops της CPU)

$N=10^8$

- Result: 333.333
- Μέσος Χρόνος: 0.0477438 sec

$N=10^6$

- Result: 333.333
- Μέσος Χρόνος: 0.0008155 sec

Ανάλυση Αποτελεσμάτων & Θεωρητική Τεκμηρίωση

1. Ο ρόλος και η αποδοτικότητα του Barrier (`__syncthreads()`)

Η εισαγωγή φραγμάτων συγχρονισμού θεωρητικά προσθέτει καθυστέρηση, καθώς αναγκάζει τα ταχύτερα νήματα ενός Warp να σταματήσουν και να περιμένουν τα πιο αργά νήματα του ίδιου block (Warp Stall Overhead). Ωστόσο, στην αρχιτεκτονική της GPU, η `__syncthreads()` είναι υλοποιημένη απευθείας στο hardware, γεγονός που ελαχιστοποιεί αυτό το overhead.

- Για μεγάλο μέγεθος ($N = 10^8$), ο καθαρός χρόνος είναι **0.0477 sec**. Αν συγκριθεί με την απλή έκδοση Global (0.0467 sec), παρατηρούμε μια ανεπαίσθητη επιβάρυνση της τάξης του **~2%**. Αυτό αποδεικνύει ότι το υπολογιστικό κόστος του integration (Compute Bound) κυριαρχεί, "κρύβοντας" επιτυχώς το hardware κόστος του συγχρονισμού.
- Για μικρό μέγεθος ($N = 10^6$), ο χρόνος πέφτει στα **0.00081 sec**. Σε αυτό το επίπεδο, το καθαρό υπολογιστικό έργο μειώνεται δραματικά. Το overhead του συγχρονισμού και των barriers παραμένει σταθερό ανά block, με αποτέλεσμα να αποτελεί μεγαλύτερο ποσοστό του συνολικού χρόνου εκτέλεσης, χωρίς όμως να υποβαθμίζει την απόδοση της κάρτας.

2. Διασφάλιση Ορθότητας Δεδομένων (Race Conditions)

Η χρήση της `__syncthreads()` σε κάθε επίπεδο του reduction loop (`step != 2`) είναι ο μοναδικός τρόπος για να εξασφαλιστεί η **ατομικότητα και η ορθότητα των υπολογισμών**. Χωρίς το barrier, κάποια νήματα θα επιχειρούσαν να διαβάσουν δεδομένα από τη shared memory (`cache[tid + step]`) προτού το υπεύθυνο νήμα προλάβει να γράψει την ενημερωμένη τιμή, οδηγώντας σε λανθασμένο τελικό αποτέλεσμα. Η επίτευξη του απόλυτα ακριβούς αποτελέσματος 333.333 και στις δύο δοκιμές επιβεβαιώνει τη σωστή τοποθέτηση των barriers.

Σύγκριση των δύο περιπτώσεων

- Στο μεγάλο N, το κόστος του συγχρονισμού “κρύβεται” πίσω από τον μεγάλο αριθμό υπολογισμών
- Στο μικρό N, ο συγχρονισμός και το kernel overhead έχουν πιο έντονη επίδραση στον συνολικό χρόνο
- Και στις δύο περιπτώσεις, το αποτέλεσμα παραμένει αριθμητικά σωστό (333.333)

Συμπέρασμα

Από την πειραματική μελέτη της έκδοσης με Barriers προκύπτουν τα εξής συμπεράσματα:

- **Αναγκαίο Κόστος:** Ο συγχρονισμός μέσω `__syncthreads()` είναι αρχιτεκτονικά απαραίτητος όταν υπάρχει διαμοιρασμός δεδομένων και επικοινωνία μεταξύ νημάτων (όπως στο Parallel Reduction μέσω Shared Memory).
- **Hardware Efficiency:** Τα barriers στις σύγχρονες GPU είναι εξαιρετικά ελαφριά (lightweight) και δεν «πνίγουν» τον kernel, ειδικά σε μεγάλα προβλήματα (10^8) όπου το computational density είναι υψηλό.
- **Συνολικό Κέρδος:** Παρόλο που ο καθαρός χρόνος του kernel είναι ελάχιστα μεγαλύτερος από την απλή Global έκδοση λόγω των αναμονών, η χρήση shared memory με barriers μειώνει δραματικά τα τελικά global writes κατά ένα παράγοντα ίσο με το μέγεθος του block (1/256). Αυτό αποσυμφορεί πλήρως το PCIe bus και εκμηδενίζει το χρόνο του reduction που θα εκτελούσε αργότερα η CPU.

Οδηγίες Εκτέλεσης

- **Compile:** `nvcc trapezoid_barrier.cu -o trapezoid_barrier`

- **Run:** .\trapezoid_barrier.exe

Μοντέλα μνήμης (registers, global, shared)

Στην παρούσα ενότητα εξετάζεται η επίδραση του μοντέλου μνήμης στην απόδοση ενός CUDA kernel για τον υπολογισμό αριθμητικού ολοκληρώματος με τη μέθοδο των τραπεζίων. Σε όλες τις υλοποιήσεις χρησιμοποιείται το ίδιο μοντέλο κατανομής εργασίας (**1 thread = 1 τραπέζιο**), ώστε η σύγκριση να βασίζεται αποκλειστικά στον τρόπο διαχείρισης και προσπέλασης των διαφορετικών επιπέδων μνήμης της GPU.

Εξετάζονται τρεις διαφορετικές προσεγγίσεις:

- χρήση **global memory**
- χρήση **registers (local variables)**
- χρήση **shared memory με συγχρονισμό (barriers)**

Εξετάστηκαν δύο μεγέθη προβλήματος:

- $N=10^8$
- $N=10^6$

Αποτελέσματα εκτέλεσης

Για $N=10^8$

Μοντέλο Μνήμης	Καθαρός Μέσος Χρόνος Εκτέλεσης (sec)	Αποτέλεσμα Ολοκληρώματος
Global	0.0457837	333.333
Register	0.0461351	333.333
Shared	0.0477410	333.333

Για $N=10^6$

Μοντέλο Μνήμης	Καθαρός Μέσος Χρόνος Εκτέλεσης (sec)	Αποτέλεσμα Ολοκληρώματος
Global	0.0007993	333.333

Register	0.0007925	333.333
Shared	0.0008210	333.333

Ανάλυση Αποτελεσμάτων & Θεωρητική Τεκμηρίωση

1. Η συμπεριφορά Global Memory και Registers (Compiler Optimization)

Από τα αποτελέσματα των μετρήσεων γίνεται άμεσα εμφανές ότι οι υλοποιήσεις **Global** και **Register** επιτυγχάνουν ουσιαστικά την ίδια ακριβώς επίδοση (~0.045 - 0.046 sec για το μεγάλο N και ~0.00079 sec για το μικρό N).

- **Θεωρητική εξήγηση:** Σύγχρονες αρχιτεκτονικές GPU και οι προηγμένοι compilers (nvcc) πραγματοποιούν αυτόματη βελτιστοποίηση επιπέδου καταχωρητών (Instruction-Level Optimization). Ακόμα και στην απλή έκδοση Global, ο compiler δεν εκτελεί τις αριθμητικές πράξεις απευθείας πάνω στη Global μνήμη (κάτι που θα ήταν καταστροφικό λόγω latency), αλλά δεσμεύει προσωρινά registers. Συνεπώς, σε επίπεδο παραγόμενου κώδικα hardware, οι δύο kernels καταλήγουν να είναι σχεδόν ταυτόσημοι.

2. Η επίδραση της Shared Memory και το Barrier Overhead

Η έκδοση με **Shared Memory** παρουσιάζει ελάχιστα μεγαλύτερο χρόνο εκτέλεσης (0.0477 sec έναντι 0.0457 sec στο μεγάλο πρόβλημα).

- **Θεωρητική εξήγηση:** Στο συγκεκριμένο μοντέλο κατανομής εργασίας (1 thread = 1 στοιχείο), η χρήση της Shared μνήμης για το reduction εισάγει την ανάγκη για συστηματικό συγχρονισμό των νημάτων μέσω της εντολής `__syncthreads()`. Αυτά τα hardware barriers αναγκάζουν τα ταχύτερα νήματα να περιμένουν τα πιο αργά νήματα του ίδιου block (Warp divergence / Synchronization stalls).
- Ωστόσο, αυτή η μικρή καθυστέρηση στον καθαρό χρόνο του kernel είναι απολύτως αποδεκτή, καθώς η Shared Memory **μειώνει τον όγκο των τελικών δεδομένων** που επιστρέφουν στον Host (CPU) κατά ένα παράγοντα ίσο με το `block_size`, εξαλείφοντας το bottleneck του PCIe bus και της CPU αργότερα.

Σύγκριση των μοντέλων

Μοντέλο Μνήμης	Απόδοση	Παρατηρήσεις (Θεωρητική Τεκμηρίωση)
Global	Μέγιστη	Αξιοποιεί το Memory Coalescing (συνενωμένες προσβάσεις). Ο compiler βελτιστοποιεί τις πράξεις χρησιμοποιώντας registers στο παρασκήνιο.
Register	Ταυτόσημη	Επιβεβαιώνει την έξυπνη διαχείριση του compiler. Δεν υπάρχει κέρδος από τη χειροκίνητη δήλωση τοπικών μεταβλητών σε τόσο απλούς kernels.
Shared	Υψηλή	Εισάγει ένα απειροελάχιστο overhead λόγω των barriers (<code>__syncthreads()</code>), αλλά προσφέρει δραστική μείωση των Global Writes προς όφελος της CPU.

Συμπεράσματα

- **Compiler Intelligence:** Οι δοκιμές αποδεικνύουν ότι ο compiler του CUDA είναι εξαιρετικά ικανός στο να προστατεύει τον προγραμματιστή από άσκοπα memory stalls, βελτιστοποιώντας αυτόματα τη χρήση των registers στην Global έκδοση.
- **Στρατηγική Επιλογή:** Παρά το ανεπαίσθητο overhead συγχρονισμού της Shared Memory (~4%), αυτή παραμένει η βέλτιστη αρχιτεκτονική προσέγγιση για end-to-end εφαρμογές, διότι επιτρέπει στα νήματα να συνεργάζονται on-chip, ελαχιστοποιώντας το τελικό αποτύπωμα μνήμης προς τη CPU.

Οδηγίες Εκτέλεσης

- **Compile:**
 - `nvcc memory_global.cu -o memory_global`
 - `nvcc memory_register.cu -o memory_register`
 - `nvcc memory_shared.cu -o memory_shared`
- **Run:**
 - `.\memory_global.exe`
 - `.\memory_register.exe`
 - `.\memory_shared.exe`

Τελική Βέλτιστη Υλοποίηση

Από τη συνολική πειραματική ανάλυση των διαφορετικών στρατηγικών παραλληλοποίησης προκύπτει ότι η βέλτιστη απόδοση επιτυγχάνεται με συνδυασμό των εξής επιλογών:

- Η πιο αποδοτική υλοποίηση είναι αυτή που χρησιμοποιεί **1 thread = πολλαπλά στοιχεία (grid-stride loop)**, καθώς μειώνει δραστικά τον αριθμό των global memory writes και αξιοποιεί καλύτερα τους registers για την αποθήκευση ενδιάμεσων αποτελεσμάτων.
- Παράλληλα, η χρήση **thread block μεγέθους 128 έως 256 threads** φαίνεται να προσφέρει την καλύτερη ισορροπία μεταξύ occupancy και latency hiding, ειδικά για μεγάλα προβλήματα ($N = 10^8$), όπου η GPU είναι πλήρως φορτωμένη.
- Τέλος, η ενσωμάτωση **shared memory με συγχρονισμό (__syncthreads())** είναι απαραίτητη μόνο σε περιπτώσεις όπου απαιτείται συνεργατική επεξεργασία (π.χ. reduction), ενώ για απλό υπολογισμό ανά element η χρήση register-based τοπικών μεταβλητών παραμένει πιο αποδοτική.

Συνεπώς, η βέλτιστη αρχιτεκτονική για το συγκεκριμένο πρόβλημα είναι:

Grid-stride work distribution + block size 128-256 + χρήση registers για intermediate values + περιορισμένη χρήση shared memory μόνο όπου απαιτείται συγχρονισμός.

Αυτή η συνδυαστική προσέγγιση προσφέρει τη μέγιστη απόδοση, τη βέλτιστη κλιμάκωση ως προς το μέγεθος του προβλήματος και την καλύτερη αξιοποίηση της αρχιτεκτονικής της GPU.

Ποσοτική σύγκριση βέλτιστων υλοποιήσεων (GPU vs OpenMP vs POSIX)

Πραγματοποιείται ποσοτική σύγκριση της βέλτιστης υλοποίησης με GPU (CUDA) με τις βέλτιστες υλοποιήσεις που αναπτύχθηκαν στις 1η και 2η προγραμματιστική εργασία (POSIX threads και OpenMP) για το ίδιο πρόβλημα αριθμητικής ολοκλήρωσης με τον κανόνα των τραπεζίων.

Για δίκαιη σύγκριση χρησιμοποιείται το

- ίδιο μέγεθος προβλήματος $N = 10^8$
- ίδια $f(x)=x^2$.

Βέλτιστη GPU υλοποίηση (CUDA)

Από την πειραματική ανάλυση προκύπτει ότι η βέλτιστη GPU υλοποίηση είναι:

Πολλά στοιχεία ανά thread (1 thread = πολλαπλά trapezoids)

- Χρόνος εκτέλεσης: 0.0257379 sec ($N = 10^8$)
 - ΟΠΟΥ ΠΕΡΙΛΑΜΒΑΝΕΙ ΚΑΙ GPU & CPU TIME

Βέλτιστη OpenMP υλοποίηση

Η βέλτιστη OpenMP εκδοχή βασίζεται σε:

Parallel for χωρίς reduction

- Χρόνος εκτέλεσης: 0.1077 sec ($N = 10^8$)

Βέλτιστη POSIX Threads υλοποίηση

Η βέλτιστη POSIX εκδοχή βασίζεται σε:

Δυναμικό διαμοιρασμό μέσω ουράς εργασιών

Χρόνος εκτέλεσης: 0.0744 sec ($N = 10^8$)

Πίνακας σύγκρισης

Υλοποίηση	Χρόνος (sec)	Speedup vs GPU
CUDA	0.0257379	-
OpenMP	0.1077	4.18x (0.1077/0.0257379)
POSIX	0.0744	2.89x (0.0744/0.0257379)

Ανάλυση αποτελεσμάτων

1. GPU (CUDA)

Η GPU υλοποίηση υπερέχει λόγω:

- μαζικού parallelism
- ελάχιστα global memory writes
- υψηλό occupancy

2. OpenMP

Η OpenMP υλοποίηση:

- αξιοποιεί CPU cores (6 στον Ryzen 5 5500)

- έχει χαμηλό overhead σε σχέση με pthreads

3. POSIX Threads

Η POSIX υλοποίηση:

- έχει μεγαλύτερο thread management overhead
- απαιτεί χειροκίνητο synchronization
- δεν έχει compiler-level optimizations όπως OpenMP

Τελικό συμπέρασμα

Η σύγκριση δείχνει ότι:

- Η **CUDA GPU υλοποίηση είναι κατά τάξεις μεγέθους ταχύτερη** από τις CPU-based προσεγγίσεις.
- Η διαφορά προκύπτει από:
 - μαζικό parallelism (GPU warps vs CPU cores)
 - αποφυγή memory bottlenecks
 - βελτιστοποιημένο workload